

PyTorch — Core Concepts

Tensors • nn.Module • Training Loop • Dataset & DataLoader

Part 1 — Tensors & Operations

A tensor is the fundamental data structure in PyTorch. Think of it as a multi-dimensional array — like a NumPy array but with two superpowers: it can live on a GPU, and it can track gradients for automatic differentiation.

1.1 Creating Tensors

There are several ways to create tensors:

```
import torch

# From Python lists
t = torch.tensor([1, 2, 3])          # 1D tensor
t = torch.tensor([[1,2],[3,4]])      # 2D tensor (matrix)

# Filled tensors
torch.zeros(3, 4)                   # 3x4 matrix of zeros
torch.ones(2, 3)                    # 2x3 matrix of ones
torch.full((2, 3), 7)               # 2x3 matrix of 7s
torch.eye(3)                        # 3x3 identity matrix

# Random tensors
torch.rand(3, 4)                    # uniform [0, 1)
torch.randn(3, 4)                   # normal distribution (mean=0, std=1)
torch.randint(0, 10, (3, 4))        # random ints between 0 and 9

# From a range
torch.arange(0, 10, 2)               # [0, 2, 4, 6, 8]
torch.linspace(0, 1, 5)              # [0, 0.25, 0.5, 0.75, 1.0]

# Copy shape of existing tensor
torch.zeros_like(t)                 # same shape as t, filled with 0
torch.ones_like(t)
```

1.2 Tensor Attributes

```
t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])

t.shape          # torch.Size([2, 2])
t.dtype          # torch.float32
t.device         # device(type='cpu')
t.requires_grad  # False by default
```

dtype options: torch.float32 (default for floats), torch.float16, torch.float64, torch.int32, torch.int64, torch.bool.

1.3 Tensor Operations

Math operations

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

a + b            # element-wise add   → [5, 7, 9]
a * b            # element-wise mul   → [4, 10, 18]
torch.dot(a, b)  # dot product        → 32.0
torch.matmul(A, B) # matrix multiply  (also A @ B)
a.sum()          # sum of all elements
a.mean()
a.max(), a.min()
a.abs()
torch.sqrt(a)
```

Reshaping

```
t = torch.arange(12)      # [0,1,2,...,11]
t.reshape(3, 4)           # 3 rows, 4 cols
t.reshape(3, -1)          # -1 means 'infer this dim'
t.view(2, 6)              # like reshape but shares memory
t.unsqueeze(0)            # add dim at position 0: [12] → [1, 12]
t.squeeze()              # remove all size-1 dimensions
t.permute(1, 0)           # transpose: swap dim 0 and 1
```

Indexing & Slicing

```
t = torch.tensor([[1,2,3],[4,5,6]])
t[0]                  # first row → tensor([1, 2, 3])
t[:, 1]              # second column → tensor([2, 5])
t[0, 2]              # single element → tensor(3)
t[t > 3]             # boolean indexing → tensor([4, 5, 6])
```

Concatenating tensors

```
torch.cat([a, b], dim=0) # join along existing dim
torch.stack([a, b], dim=0) # create new dim and stack
```

1.4 GPU vs CPU Tensors

By default, all tensors are on CPU. To use GPU you move them to a CUDA device. This is one of the most common sources of errors in PyTorch — you can't do operations between a CPU tensor and a GPU tensor.

```
# Check if GPU is available
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# Move tensor to GPU
t = torch.tensor([1.0, 2.0])
t = t.to(device)          # move to GPU
t = t.to('cuda')          # explicit GPU
t = t.to('cpu')           # move back to CPU
t = t.cuda()              # shorthand

# Create directly on GPU
t = torch.zeros(3, 4, device='cuda')

# This will ERROR — different devices
a = torch.tensor([1.0])   # CPU
b = torch.tensor([2.0]).cuda() # GPU
a + b # RuntimeError: expected all tensors on same device
```

Key difference: CPU tensors use RAM and are computed on CPU cores. GPU tensors use VRAM and are computed on GPU cores in parallel. For large tensor operations (like neural network layers), GPU is orders of magnitude faster. For tiny tensors, the overhead of GPU data transfer can make CPU faster.

Converting between tensor and NumPy

```
import numpy as np

# Tensor → NumPy (CPU only)
t = torch.tensor([1.0, 2.0])
arr = t.numpy()          # shares memory with t
arr = t.detach().numpy()  # if t has grad
arr = t.cpu().detach().numpy() # if t is on GPU

# NumPy → Tensor
t = torch.from_numpy(arr) # shares memory
t = torch.tensor(arr)     # copies data
```

Part 2 — Neural Networks with nn.Module

nn.Module is the base class for all neural networks in PyTorch. Every model you build inherits from it. It gives you automatic parameter tracking, a clean way to define layers, and hooks into the training/evaluation system.

2.1 Basic Structure

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__() # always call this first
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        x = self.fc1(x) # (batch, 784) → (batch, 128)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x) # (batch, 128) → (batch, 10)
        return x

model = MyModel()
```

Two mandatory things: `__init__` (define all layers) and `forward` (define how data flows through them). The backward pass is handled automatically by PyTorch's autograd.

2.2 Common Layers

Linear layers

```
nn.Linear(in_features, out_features, bias=True)
# Does: output = input @ weight.T + bias
# input shape: (batch, in_features)
# output shape: (batch, out_features)
```

Activation functions

```
nn.ReLU() # max(0, x) – most common for hidden layers
nn.Sigmoid() # (0, 1) – binary classification output
nn.Tanh() # (-1, 1)
nn.GELU() # used in transformers (smoother than ReLU)
nn.Softmax(dim=1) # probability distribution – multi-class output
nn.LeakyReLU(0.01) # allows small negative values
```

Normalization

```
nn.BatchNorm1d(num_features) # for 1D/tabular data
nn.BatchNorm2d(num_channels) # for images (CNN)
nn.LayerNorm(normalized_shape) # for transformers
nn.Dropout(p=0.5) # randomly zeroes p% of neurons
```

Convolutional layers

```
nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)
nn.MaxPool2d(kernel_size, stride)
nn.AvgPool2d(kernel_size)
```

Embedding

```
nn.Embedding(num_embeddings, embedding_dim)
# converts token indices → dense vectors
# e.g. nn.Embedding(vocab_size=10000, embedding_dim=256)
```

2.3 Model Parameters & Device

```
model = MyModel()

# Count parameters
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)

# Move model to GPU – moves ALL parameters
model = model.to(device)
```

```

model = model.cuda()

# Save / load
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))

```

2.4 Train vs Eval Mode

This is critical — always remember to switch modes. Certain layers (BatchNorm, Dropout) behave differently during training and inference.

```

model.train()    # enables dropout, batch norm uses batch stats
model.eval()     # disables dropout, batch norm uses running stats

# During inference, also wrap in no_grad to save memory
with torch.no_grad():
    output = model(input_tensor)

```

Dropout: during training, randomly zeroes neurons for regularization. During eval, all neurons are active and outputs are scaled.

BatchNorm: during training, uses mean/std of the current batch. During eval, uses running mean/std accumulated during training.

2.5 Sequential and ModuleList

```

# For simple sequential models - shorter syntax
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)

# ModuleList - for dynamic number of layers
class DeepNet(nn.Module):
    def __init__(self, num_layers):
        super().__init__()
        self.layers = nn.ModuleList(
            [nn.Linear(128, 128) for _ in range(num_layers)]
        )

    def forward(self, x):
        for layer in self.layers:
            x = torch.relu(layer(x))
        return x

```

Important: use `nn.ModuleList` / `nn.ModuleDict` instead of plain Python lists when storing layers — otherwise PyTorch won't register them as parameters and they won't be saved or moved to GPU.

Part 3 — Training Loop

The training loop is where the model actually learns. In PyTorch, unlike Keras, the loop is written by you explicitly — you control every step. This gives full flexibility but means you must remember each piece.

3.1 The Five Steps

Every training iteration has exactly five steps:

- **1. Forward pass:** pass input through the model to get predictions.
- **2. Compute loss:** compare predictions to targets using a loss function.
- **3. Zero gradients:** clear old gradients (they accumulate by default in PyTorch).
- **4. Backward pass:** compute gradients via backpropagation.
- **5. Optimizer step:** update weights using the computed gradients.

3.2 Full Training Loop

```

import torch
import torch.nn as nn

model = MyModel().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    model.train()  # set to train mode

    for batch_X, batch_y in train_loader:
        batch_X = batch_X.to(device)
        batch_y = batch_y.to(device)

        # 1. Forward pass
        predictions = model(batch_X)

        # 2. Compute loss
        loss = criterion(predictions, batch_y)

        # 3. Zero gradients — MUST do this before backward
        optimizer.zero_grad()

        # 4. Backward pass
        loss.backward()

        # 5. Update weights
        optimizer.step()

    print(f'Epoch {epoch}: Loss = {loss.item():.4f}')

```

3.3 Loss Functions

- **nn.CrossEntropyLoss()**: multi-class classification. Combines LogSoftmax + NLLLoss internally. Expects raw logits (not softmax output).
- **nn.BCEWithLogitsLoss()**: binary classification. Combines Sigmoid + BCE. More numerically stable than applying Sigmoid then BCE separately.
- **nn.MSELoss()**: regression. Mean Squared Error — $(\text{pred} - \text{target})^2$.
- **nn.L1Loss()**: regression. Mean Absolute Error — $|\text{pred} - \text{target}|$. Less sensitive to outliers than MSE.
- **nn.NLLLoss()**: negative log likelihood — use with `log_softmax` output.

3.4 Optimizers

```

# SGD — classic, needs careful learning rate tuning
torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# Adam — adaptive learning rate, usually the go-to
torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

# AdamW — Adam with decoupled weight decay, preferred for transformers
torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.01)

# RMSprop — good for recurrent networks
torch.optim.RMSprop(model.parameters(), lr=1e-3)

```

`weight_decay` adds L2 regularization — penalizes large weights to prevent overfitting.

3.5 Learning Rate Schedulers

Learning rate often needs to decay over training for convergence. PyTorch has built-in schedulers:

```

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
# ↑ reduces LR by factor 0.1 every 10 epochs

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
# ↑ cosine decay — common in modern training

```

```

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5)
# ↑ reduces LR when val loss stops improving

# Call at end of each epoch:
scheduler.step()

```

3.6 Validation Loop

Always evaluate on validation data after each epoch to catch overfitting:

```

model.eval()
val_loss = 0
correct = 0

with torch.no_grad():
    # no gradient computation
    for batch_X, batch_y in val_loader:
        batch_X, batch_y = batch_X.to(device), batch_y.to(device)
        predictions = model(batch_X)
        val_loss += criterion(predictions, batch_y).item()
        pred_classes = predictions.argmax(dim=1)
        correct += (pred_classes == batch_y).sum().item()

accuracy = correct / len(val_loader.dataset)
print(f'Val Loss: {val_loss:.4f}, Accuracy: {accuracy:.4f}')

```

3.7 Gradient Clipping

Useful for RNNs/transformers where gradients can explode. Clip before optimizer.step():

```

loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

```

Part 4 — Dataset & DataLoader

PyTorch separates data handling into two classes: Dataset (what the data is) and DataLoader (how to serve it to the model in batches). You almost always need to implement a custom Dataset.

4.1 The Dataset Class

To make a custom dataset, inherit from `torch.utils.data.Dataset` and implement exactly three methods:

```

from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.X)  # total number of samples

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]  # one sample by index

dataset = MyDataset(X_train, y_train)
print(len(dataset))  # calls __len__
print(dataset[0])    # calls __getitem__ with idx=0

```

Why these three methods?

- `__init__`: load or reference your data. Usually store file paths or the actual data here.
- `__len__`: DataLoader needs to know total size to plan batches and shuffling.
- `__getitem__`: DataLoader calls this with random indices to build each batch. This is where transforms, augmentations, or file reads happen.

4.2 Image Dataset Example

```
import os
from PIL import Image
from torchvision import transforms

class ImageDataset(Dataset):
    def __init__(self, image_dir, labels, transform=None):
        self.image_paths = [os.path.join(image_dir, f) for f in os.listdir(image_dir)]
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert('RGB')
        label = self.labels[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

4.3 DataLoader

DataLoader wraps a Dataset and handles batching, shuffling, and parallel loading.

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,           # shuffle at start of each epoch
    num_workers=4,         # parallel data loading workers
    pin_memory=True,       # faster CPU→GPU transfer
    drop_last=True         # drop incomplete last batch
)

val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
# Note: never shuffle validation data
```

Key DataLoader arguments

- **batch_size**: how many samples per batch. Larger = faster training but more VRAM. Common values: 16, 32, 64, 128.
- **shuffle=True**: randomize order each epoch. Essential for training — prevents the model from learning data order. Never shuffle validation/test.
- **num_workers**: parallel processes for loading data. Set to number of CPU cores (typically 4-8). Set to 0 on Windows if getting errors.
- **pin_memory=True**: pre-loads batches into pinned (page-locked) CPU memory for faster GPU transfer. Use when training on GPU.
- **drop_last=True**: drops the last batch if it's smaller than batch_size. Useful when using BatchNorm (it breaks with batch size 1).

4.4 Train / Val / Test Split

```
from torch.utils.data import random_split

dataset = MyDataset(X, y)
n = len(dataset)
```

```

train_size = int(0.8 * n)
val_size   = int(0.1 * n)
test_size  = n - train_size - val_size

train_ds, val_ds, test_ds = random_split(dataset, [train_size, val_size, test_size])

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_ds,   batch_size=64, shuffle=False)
test_loader  = DataLoader(test_ds,  batch_size=64, shuffle=False)

```

4.5 Iterating the DataLoader

```

# In training loop
for batch_X, batch_y in train_loader:
    print(batch_X.shape)    # (32, num_features) - batch of 32
    print(batch_y.shape)    # (32,)
    # ... training steps

# One-shot peek at a batch
batch_X, batch_y = next(iter(train_loader))

```

Interview Questions

Tensors

Q1. What is the difference between `torch.tensor()` and `torch.Tensor()`?

Answer: `torch.tensor(data)` infers dtype from the data and makes a copy. `torch.Tensor()` is the class constructor — `torch.Tensor(3,4)` creates an uninitialized 3x4 float tensor. In practice always use `torch.tensor()` with explicit dtype for predictability.

Q2. What's the difference between `.view()` and `.reshape()`?

Answer: `.view()` requires the tensor to be contiguous in memory — it returns a view (shares memory, no copy). `.reshape()` works even on non-contiguous tensors — it returns a view if possible, otherwise copies. In practice, use `.reshape()` to be safe. If you need guaranteed memory sharing, use `.view()`.

Q3. What does `tensor.detach()` do? When do you need it?

Answer: `.detach()` creates a new tensor that shares data but is detached from the computation graph — no gradients flow through it. You need it when: converting a tensor to NumPy (`t.detach().numpy()`), logging intermediate values during training without affecting backprop, or when you want to use a tensor's value as a constant in a loss calculation.

Q4. Why can't you do operations between a CPU tensor and a GPU tensor?

Answer: CPU and GPU have separate memory spaces. A CPU tensor lives in RAM, a GPU tensor lives in VRAM. To do math between them, data would need to be transferred across the PCIe bus every operation, which would be extremely slow and PyTorch would have to guess where to do the computation. Instead, PyTorch throws a `RuntimeError` so you explicitly move everything to the same device first.

Q5. What is `torch.no_grad()` and when should you use it?

Answer: A context manager that disables gradient tracking for all operations inside it. Use during inference (model evaluation) and validation loops. Without it, PyTorch builds a computation graph for every operation to support backprop — this uses extra memory and time that's wasted when you don't need gradients. Rule: always wrap eval loops with `torch.no_grad()`.

nn.Module

Q6. What are the two mandatory methods when subclassing `nn.Module`?

Answer: `__init__` (define all layers — must call `super().__init__()` first) and `forward` (define how data flows through the layers). The backward pass is never written manually — PyTorch autograd computes it automatically from the forward pass operations.

Q7. Why must you use `nn.ModuleList` instead of a plain Python list for storing layers?

Answer: `nn.Module` only tracks layers that are registered as module attributes. A plain Python list is invisible to PyTorch — its contents won't appear in `model.parameters()`, won't be saved with `state_dict()`, and won't be moved to GPU with `model.to(device)`. `nn.ModuleList` is a proper PyTorch container that registers all its contents as submodules.

Q8. What is the difference between `model.train()` and `model.eval()`?

Answer: `model.train()` enables training behaviour: Dropout randomly zeroes neurons, BatchNorm uses batch statistics. `model.eval()` disables Dropout (all neurons active) and BatchNorm uses running statistics accumulated during training. Forgetting to call `model.eval()` before inference is a common bug — you get different outputs each time due to Dropout being active.

Q9. How do you freeze layers in PyTorch (e.g. for transfer learning)?

Answer: Set `requires_grad=False` on the parameters you want to freeze. They'll still do the forward pass but won't accumulate gradients and the optimizer won't update them.

```
for param in model.base_model.parameters():
    param.requires_grad = False

# Only train the new head
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters())
)
```

Training Loop

Q10. Why do you need to call `optimizer.zero_grad()` before `loss.backward()`?

Answer: In PyTorch, gradients accumulate — each call to `.backward()` adds gradients to the existing `.grad` values rather than replacing them. This is useful for things like gradient accumulation over multiple mini-batches, but for a standard training loop you want fresh gradients each step. If you forget `zero_grad()`, gradients from previous batches pile up and your weight updates become wrong.

Q11. What is the correct order of the five training steps?

Answer: 1) forward pass: `predictions = model(X)`, 2) compute loss: `loss = criterion(predictions, y)`, 3) zero gradients: `optimizer.zero_grad()`, 4) backward pass: `loss.backward()`, 5) update weights: `optimizer.step()`. Steps 3 and 4 must come in this order — you need to clear old gradients before computing new ones.

Q12. When would you use `CrossEntropyLoss` vs `BCEWithLogitsLoss`?

Answer: `CrossEntropyLoss` for multi-class classification (output is one class out of N — e.g. digit 0-9). `BCEWithLogitsLoss` for binary classification or multi-label classification (each output independently is 0 or 1). Both expect raw logits (before softmax/sigmoid) — applying activation before these loss functions causes numerical instability.

Q13. What is gradient clipping and why is it used?

Answer: Gradient clipping caps the gradient norm at a max value before the optimizer step. Used to prevent exploding gradients — a common problem in RNNs and transformers where long sequences cause gradients to grow exponentially during backprop. `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` is called between `loss.backward()` and `optimizer.step()`.

Dataset & DataLoader

Q14. What are the three methods you must implement in a custom Dataset class?

Answer: `__init__` (initialize data, file paths, transforms), `__len__` (return total number of samples — DataLoader uses this to plan batches), `__getitem__` (return one sample given an index — DataLoader calls this with shuffled

indices to build each batch). This interface lets DataLoader work with any data source without knowing its internal structure.

Q15. Why should shuffle=True for training but shuffle=False for validation?

Answer: During training, shuffling prevents the model from learning patterns based on data order (e.g. if all cats come before all dogs, the model might learn to predict based on position). For validation and testing, you never shuffle — you want reproducible, deterministic results so you can compare evaluations meaningfully across runs.

Q16. What does num_workers do in DataLoader and what happens if you set it too high?

Answer: num_workers spawns that many parallel worker processes to load data while the GPU is training — data loading happens in the background so the GPU isn't waiting. Too high can cause issues: on Windows, multiprocessing can crash (set to 0 on Windows). In general, setting it to the number of CPU cores (4-8) is a good default. If you run out of shared memory (especially in Docker/Colab), reduce to 0 or 2.

Q17. What is pin_memory=True and when should you use it?

Answer: pin_memory=True allocates DataLoader output tensors in pinned (page-locked) CPU memory instead of regular pageable memory. Pinned memory can be transferred to GPU memory much faster via DMA (direct memory access). Use it whenever you're training on GPU. Has no effect when training on CPU. The tradeoff is it uses more RAM — avoid if CPU memory is tight.

Q18. How would you handle a dataset too large to fit in RAM?

Answer:

- In __getitem__, load each file from disk on demand instead of loading everything in __init__
- Use memory-mapped files (np.memmap or HDF5 with h5py) for fast partial reads
- Use streaming datasets (HuggingFace datasets with streaming=True)
- Pre-process and save to sharded binary files, load one shard at a time
- num_workers > 0 so data loading runs in parallel while GPU trains